

Guion de exposición · Bases de datos vectoriales

Versión ampliada con fundamentos teóricos, ejemplos resueltos y aclaraciones para preguntas. Cinco integrantes; 15 diapositivas.

Cómo usar este guion ampliado

Cada diapositiva contiene un guion oral, con las acciones y transiciones de la exposición, seguido por la base teórica para estudiar y explicar con palabras propias. Los ejemplos resueltos y las precisiones son apoyo: seleccionarlos durante el ensayo según las dudas del público.

El recorrido oral conserva un presupuesto de 26 minutos, con hasta 4 minutos de margen. Leer también todas las ampliaciones excedería ese tiempo. Las interacciones están incluidas en el presupuesto; no se suman después. La duración debe comprobarse con un ensayo grupal. Asignar los nombres reales a los roles antes de exponer.

Para preparar cada tema: leer el fundamento, explicar el ejemplo sin mirar y ensayar el bloque oral con el gráfico. Todos deben poder distinguir vector, embedding, índice y base de datos; similitud y relevancia; filtros y ranking; recuperación y generación.

Las referencias [L1-L3] remiten al informe, al código y a los resultados locales. [T1-T8] identifican fuentes teóricas y documentación, listadas al final. Las diapositivas conservan sus notas breves; este guion incorpora el desarrollo para estudiar.

Reparto y reloj acumulado

Integrante 1 · diapositivas 1–3 · 00:00–04:00. Problema y definición.

Integrante 2 · diapositivas 4–6 · 04:00–09:00. Representación, coseno y vecinos; interacción 1.

Integrante 3 · diapositivas 7–9 · 09:00–15:00. HNSW, filtros y administración; interacciones 3 y 2.

Integrante 4 · diapositivas 10–12 · 15:00–20:00. Arquitectura, aplicaciones y RAG.

Integrante 5 · diapositivas 13–15 · 20:00–26:00. Buscador, resultados y conclusión.

Preparación de la mesa de exposición

Abrir Bases_Vectoriales_Exposicion_Final.pptx en modo presentación y dejar la aplicación en otra ventana. El PDF de diapositivas contiene las mismas 15 páginas como respaldo. Este PDF del guion es un documento distinto, ampliado para preparar la explicación.

Desde la carpeta del proyecto ejecutar: `.\venv\Scripts\python.exe -m streamlit run app.py`

Abrir la dirección que indique Streamlit. La página empieza en Buscador. Seleccionar Exposición interactiva para acceder al laboratorio; el botón Ir al caso práctico: buscador permite volver. En una pantalla de proyección se puede contraer la barra lateral para dar más espacio al gráfico.

El laboratorio no utiliza red ni consulta el índice. Laboratorio_Vectorial_OFFLINE.html permite abrir las mismas tres interacciones directamente en un navegador aunque Streamlit no esté disponible. Ese archivo independiente no contiene el buscador real.

Antes de exponer, activar una vez el buscador con el corpus principal y hacer la consulta de prueba, para cargar modelo e índice. La primera descarga de MiniLM requiere conexión si el equipo no lo tiene. Mantener el generador en none. No cambiar los datos ni reconstruir la colección durante la exposición.

Una persona opera teclado y cambio de ventanas; cada integrante indica cuándo avanzar. Los demás exponen mirando al público. No leer todas las notas literalmente: son apoyo para ensayar y explicar con sus palabras.

Regla para las demostraciones

Los gráficos A–L y el grafo de capas son didácticos. La ejecución real ocurre en el buscador de Chroma. Las imágenes de respaldo muestran estados guardados; indicarlo si se utilizan en lugar de la ejecución en vivo.

En la demo de fútbol mostrar criterios y evidencia. No explicar credenciales ni abrir el archivo .env ante el público.

Diapositiva 1 · Bases de datos vectoriales

Integrante 1 · tiempo previsto 0:40

Guion oral dentro del tiempo asignado

APERTURA. Somos cinco integrantes y vamos a explicar qué almacena una base vectorial, cómo encuentra información parecida y qué decisiones exige su administración. El buscador de jugadores es nuestro caso de aplicación. Primero vamos a entender el mecanismo y después mostrarlo funcionando. No necesitamos empezar por el modelo generativo: una base vectorial también sirve sin RAG.

TRANSICIÓN. El punto de partida es un problema de recuperación, no una tecnología de moda.

Base teórica para preparar esta diapositiva

Una base de datos vectorial es un sistema que almacena representaciones numéricas y permite recuperar registros por cercanía entre esas representaciones. Su propósito es administrar información y ofrecer consultas de similitud. El vector aporta una forma de comparar; el gestor organiza los registros, conserva sus datos y permite recuperarlos. [L1]

Conviene separar cuatro conceptos desde el comienzo. El modelo de embeddings transforma contenido en números. El índice vectorial organiza esos números para buscar

vecinos. La base de datos administra el índice junto con identificadores, metadatos y contenido. La aplicación combina todo eso con una interfaz y reglas de negocio. Un LLM generativo es un componente adicional que puede redactar una respuesta. [L1, L2]

Ejemplo para explicarlo con palabras propias

“Tenemos muchas fichas de jugadores y una persona describe qué perfil necesita. Primero representamos las fichas numéricamente. Después usamos la base para recuperar perfiles cercanos a su pedido. Finalmente mostramos los datos que justifican el resultado.”

Idea que debe quedar clara

El tema central es cómo almacenar, organizar y recuperar vectores. El fútbol permite observar ese mecanismo y RAG muestra una posible aplicación. No hace falta generar texto para que exista una búsqueda vectorial.

Diapositiva 2 · El problema de encontrar información

Integrante 1 · tiempo previsto 1:20

Guion oral dentro del tiempo asignado

Una coincidencia literal depende de las palabras escritas. Si busco automóvil, un texto que sólo dice coche puede quedar fuera de una comparación literal. Los motores de texto pueden mejorar eso con sinónimos y otros recursos: no son simplemente igualdad de cadenas. Una recuperación semántica representa el contenido de otra manera para relacionar expresiones.

En el buscador, una persona pide un delantero con regate y desborde. Esa frase expresa una intención; la posición y los valores mínimos pueden requerir controles exactos. Desde el comienzo hay que separar parecido y cumplimiento.

TRANSICIÓN. ¿Qué agrega una base vectorial para resolver esta clase de consulta?

Base teórica para preparar esta diapositiva

Recuperar información consiste en seleccionar, entre muchos registros, los que pueden servir para una necesidad. Hay consultas exactas, como buscar un identificador o exigir edad menor o igual a 25; consultas léxicas, que utilizan términos del texto; y consultas por similitud, que comparan representaciones. Son operaciones distintas y pueden coexistir. [L1]

Una búsqueda literal por “automóvil” puede omitir un documento que sólo contenga “coche”. Es un desajuste de vocabulario. Los sistemas léxicos pueden incorporar sinónimos, normalización e índices invertidos. Por eso no debe afirmarse que toda búsqueda tradicional se limita a comparar cadenas idénticas. La búsqueda semántica intenta relacionar formulaciones por medio de una representación aprendida, con posibles errores. [L1]

En nuestro caso, “delantero con regate y desborde” mezcla una intención descriptiva con requisitos que podrían expresarse en columnas. “Parecido a mi pedido” admite un grado;

“tiene posición ST” es una condición que se cumple o no. El texto de la consulta no crea por sí solo todas las restricciones del usuario. [L2]

Ejemplo para explicarlo con palabras propias

“Dos jugadores pueden tener perfiles parecidos, pero uno juega en una posición que no quiero. La similitud ayuda a encontrar candidatos; el filtro hace cumplir esa condición.”

Error que conviene evitar

Que una búsqueda devuelva resultados cuando otra devuelve cero no demuestra que los resultados sean relevantes. Hay que examinar qué se recuperó y con qué criterio.

Diapositiva 3 · Qué almacena una base vectorial

Integrante 1 · tiempo previsto 2:00

Guion oral dentro del tiempo asignado

Un registro combina identificador, vector y metadatos; también puede guardar el documento o una referencia. El ID permite reconocerlo. El vector participa en la búsqueda. La metadata permite filtrar. El documento permite verificar e interpretar. No confundir esos cuatro papeles.

Una base relacional responde muy bien a condiciones, asociaciones e integridad. Una base vectorial recupera cercanía según una métrica. Pueden complementarse; PostgreSQL, por ejemplo, admite la extensión pgvector.

Además del índice, un gestor administra colecciones, persistencia y operaciones. Las garantías dependen del producto y el despliegue: usar vectores no agrega automáticamente alta disponibilidad.

PASE AL INTEGRANTE 2. Para entender esa búsqueda necesitamos ver qué representa un vector.

Base teórica para preparar esta diapositiva

Un registro vectorial puede entenderse como una unidad con cuatro partes. El identificador distingue el registro y permite asociarlo con su origen. El vector interviene en el cálculo de semejanza. Los metadatos contienen campos útiles para condiciones exactas, como edad y posición. El documento o referencia conserva la información que una persona puede interpretar y verificar. La disposición física concreta depende del gestor. [L1]

Una colección agrupa registros que se consultan juntos con una configuración compatible. Para comparar sus vectores deben coincidir el espacio de representación y la dimensión esperada. Tener 384 números no basta si fueron producidos por modelos incompatibles. [L1, L2]

Una base relacional organiza datos mediante tablas y relaciones; permite consultas, restricciones de integridad y transacciones según el gestor. Incorporar búsqueda vectorial no elimina esas necesidades. PostgreSQL puede almacenar y consultar vectores mediante pgvector: “relacional” y “vectorial” no son categorías necesariamente excluyentes. [T5]

Un índice es una estructura de acceso; no equivale al sistema gestor completo. La persistencia, las copias de seguridad, los permisos y la concurrencia son responsabilidades adicionales cuya implementación debe verificarse. [L1]

Ejemplo para explicarlo con palabras propias

“El ID nos dice qué ficha es; el vector ayuda a encontrarla; la edad permite filtrarla; el documento permite leer por qué apareció. Guardar sólo el vector nos dejaría sin evidencia comprensible para el usuario.”

Diapositiva 4 · Vectores, embeddings y dimensiones

Integrante 2 · tiempo previsto 1:30

Guion oral dentro del tiempo asignado

Un vector es una lista ordenada de números. Un embedding es una representación que genera un modelo a partir del contenido. Sus dimensiones no suelen equivaler a atributos humanos como velocidad o regate.

En nuestro proyecto MiniLM produce 384 componentes. Eso no significa 384 características futbolísticas identificadas. Significa que cada texto se ubica en el espacio de ese modelo. Dos vectores de modelos diferentes no se vuelven comparables sólo por tener la misma longitud.

Para consultar usamos el mismo modelo de representación que para indexar. Cambiarlo requiere revisar y normalmente reconstruir el índice. El próximo gráfico es un ejemplo geométrico de dos dimensiones, no una proyección real de MiniLM.

Base teórica para preparar esta diapositiva

Un vector es una lista ordenada de componentes: $x = (x_1, x_2, \dots, x_D)$. D es la dimensión. Un punto 2D tiene dos coordenadas; un vector de 384 dimensiones tiene 384 componentes, aunque no podamos dibujar ese espacio completo. La posición de cada componente importa para las operaciones. [L1]

Un embedding es una representación del contenido producida por un modelo. En este proyecto, la función de embeddings utiliza all-MiniLM-L6-v2 y entrega vectores densos de 384 dimensiones. “Denso” indica que las componentes se almacenan como una representación compacta con muchos valores potencialmente distintos de cero. Un vector disperso, en cambio, puede tener muchísimas posiciones y pocas activas. [T1, L1]

Las dimensiones de un embedding aprendido no suelen tener nombres individuales como “regate” o “velocidad”. La información se distribuye entre componentes. No podemos leer un número aislado y atribuirle una habilidad futbolística. Tampoco se garantiza que el modelo interprete correctamente toda frase, negación o magnitud numérica. [L1, L2]

Un texto se procesa mediante el modelo y se resume en una representación de longitud fija. La ficha completa puede conservarse por separado. Embedding y documento cumplen funciones diferentes; el vector no es una compresión reversible que permita reconstruir exactamente el texto. [L1]

Ejemplo y precaución

El vector inventado (velocidad, regate) sería interpretable porque nosotros definimos sus ejes. Las dos coordenadas del laboratorio son igualmente didácticas: no son las dos primeras dimensiones de MiniLM ni una proyección validada de los jugadores.

Mantener compatible el procesamiento de fichas y consultas es indispensable. Cambiar de modelo exige preparar una colección compatible y volver a generar representaciones cuando corresponda.

Diapositiva 5 · Similitud coseno

Integrante 2 · tiempo previsto 2:00

Guion oral dentro del tiempo asignado

La similitud coseno compara orientación: producto interno dividido por el producto de las normas. La distancia coseno es uno menos esa similitud. No es una probabilidad.

ABRIR INTERACCIÓN 1: Vectores y similitud. Mostrar 0 grados: similitud 1 y distancia 0. Pasar a 90 grados: similitud 0 y distancia 1. Pasar a 180 grados: similitud -1 y distancia 2. Mover el control entre estados y explicar que el cambio es continuo. Las flechas son no nulas y unitarias.

Volver a 90 grados, que coincide con el respaldo de esta diapositiva.

SI FALLA LA PÁGINA: señalar las flechas de la captura y narrar los otros dos estados.

TRANSICIÓN. Calcular cercanía entre dos vectores es sólo el primer paso; ahora hay que elegir entre muchos.

Base teórica para preparar esta diapositiva

La similitud coseno mide orientación entre dos vectores no nulos. Se calcula dividiendo el producto interno por el producto de sus longitudes. En la notación siguiente, Σ significa sumar sobre todas las componentes y $\sqrt{}$ significa raíz cuadrada. [T2]

FÓRMULA: $\cos(x, y) = \Sigma(x_i \cdot y_i) / (\sqrt{\Sigma x_i^2} \cdot \sqrt{\Sigma y_i^2})$

La norma $\|x\|$ es la longitud $\sqrt{\Sigma x_i^2}$. Normalizar consiste en dividir cada componente por esa longitud; así se obtiene un vector unitario. Para vectores unitarios, el coseno coincide con el producto interno. Multiplicar un vector por un número positivo cambia su longitud, pero no su orientación. [T2]

Cálculo resuelto para practicar

Tomemos $x = (1, 0)$ e $y = (3, 4)$. El producto interno es $1 \times 3 + 0 \times 4 = 3$. Las normas son 1 y 5. Entonces $\cos(x, y) = 3 / 5 = 0,6$. Con la convención del proyecto, distancia coseno = $1 - 0,6 = 0,4$. El ejemplo es geométrico: esos números no son valoraciones de jugadores.

Con vectores unitarios, 0° produce similitud 1 y distancia 0; 90° produce 0 y 1; 180° produce -1 y 2. Un vector nulo impide calcular coseno porque su norma es cero. Por eso el laboratorio rechaza la consulta (0, 0).

Comparación con otras medidas

La distancia euclídea es $\sqrt{\sum (x_i - y_i)^2}$: mide separación geométrica y depende de las longitudes. El producto interno sin normalizar también depende de magnitudes. Para vectores unitarios, la distancia euclídea al cuadrado es $2 \times (1 - \text{coseno})$, por lo que produce el mismo orden que coseno. Elegir una medida debe ser coherente con la representación. [L1]

Precisión para responder preguntas

El intervalo general del coseno es $[-1, 1]$. Un valor negativo indica orientación opuesta, no necesariamente significado lingüístico contrario. Un valor 0,9 no significa 90 % de acierto. Aunque se la llama “distancia coseno”, $1 - \text{coseno}$ no cumple en general todas las propiedades de una métrica matemática, como la desigualdad triangular. [L1]

Diapositiva 6 · Vecinos próximos: búsqueda exacta y ANN

Integrante 2 · tiempo previsto 1:30

Guion oral dentro del tiempo asignado

Top-k significa seleccionar los k vecinos más cercanos según la métrica. Esta comparación de tres perfiles es didáctica y utiliza distancia coseno calculada exactamente.

La búsqueda exhaustiva compara todos los elegibles; su trabajo crece con cantidad de vectores y dimensión. ANN explora menos candidatos a cambio de posibles omisiones. No son dos nociones de relevancia humana: son dos estrategias para recuperar vecinos en el mismo espacio.

Una buena recuperación geométrica no demuestra que los candidatos sean los mejores futbolistas. Primero hay que evaluar el embedding; después, si el índice aproxima bien sus vecinos.

PASE AL INTEGRANTE 3. Vamos a ver una de las estructuras usadas para esa búsqueda aproximada: HNSW.

Base teórica para preparar esta diapositiva

La búsqueda de vecinos próximos ordena vectores por cercanía a una consulta. En una búsqueda exacta, el resultado corresponde a los k mejores según la medida y el conjunto elegible. “Exacta” se refiere a ese problema geométrico; no certifica relevancia humana. ANN significa Approximate Nearest Neighbors, o vecinos próximos aproximados. Puede omitir alguno de los vecinos que encontraría la búsqueda exacta. [L1, T3]

Para un escaneo exhaustivo con N vectores de D componentes, calcular todas las distancias requiere trabajo proporcional a $N \times D$. Seleccionar los mejores agrega un costo que depende del algoritmo usado. Un índice aproximado busca evitar muchas comparaciones, pero consume memoria y tiene un costo de construcción. No se puede prometer que siempre sea más rápido: influyen N, D, filtros, hardware y configuración. [L1]

Ejemplo resuelto

Si las distancias de cuatro candidatos son $A=0,10$; $B=0,40$; $C=0,20$ y $D=0,80$, el top-2 exacto es A y C. Si ANN devuelve A y B, recuperó uno de los dos vecinos exactos: su recall de vecinos es $1/2 = 0,50$. Para saber si A y C son útiles para una persona se necesita otra evaluación, con juicios de relevancia.

Cómo explicarlo durante la exposición

“La búsqueda exacta nos sirve como referencia. La aproximada intenta recuperar casi los mismos vecinos con menos trabajo. La decisión consiste en aceptar una posible pérdida de cobertura a cambio de recursos o tiempo.”

Error que conviene evitar

k es la cantidad solicitada, no un umbral de calidad. Pedir cinco puede devolver cinco perfiles poco útiles si existen candidatos y no hay una regla adicional de abstención. La aplicación no implementa un umbral semántico que garantice una respuesta pertinente. [L2]

Diapositiva 7 · HNSW: navegación por capas

Integrante 3 · tiempo previsto 2:30

Guion oral dentro del tiempo asignado

HNSW organiza conexiones entre vectores en varios niveles. Los superiores permiten desplazamientos grandes; la capa inferior refina la búsqueda.

ABRIR INTERACCIÓN 3. Reiniciar y avanzar: A es entrada superior; D queda más cerca de Q; descendemos desde D; exploramos C y F; F mejora al candidato; descendemos desde F; E mejora la distancia a 25; D ya no la mejora.

Este recorrido usa un grafo fijo y una distancia euclídea simple. No estamos viendo los vectores internos de Chroma. En HNSW real hay listas de candidatos y parámetros como amplitud de exploración y conectividad. Aumentar exploración puede mejorar cobertura y también costo.

No afirmar que siempre devuelve el vecino exacto. El esquema muestra el mecanismo de navegación, no un benchmark.

VOLVER A LAS DIAPOSITIVAS. El siguiente paso es combinar vecinos con reglas exactas.

Base teórica para preparar esta diapositiva

HNSW significa Hierarchical Navigable Small World. Es un índice basado en un grafo de proximidad con varias capas. Los nodos representan vectores y las aristas permiten visitar otros nodos. La capa inferior contiene todos los elementos; las superiores contienen subconjuntos cada vez menores. Durante la construcción se asignan niveles de manera aleatoria, con menor probabilidad de alcanzar niveles altos. [T3]

La búsqueda comienza por un punto de entrada en una capa superior, se desplaza hacia nodos más cercanos a la consulta y desciende conservando una buena ubicación. En la

capa inferior explora candidatos para seleccionar vecinos. Este diseño facilita desplazamientos amplios antes de refinar la búsqueda. Las conexiones representan proximidad; no son relaciones como “pertenece al mismo club”. [T3]

Parámetros para responder preguntas

M controla la conectividad del grafo. Mayor conectividad suele requerir más memoria. ef_construction controla la amplitud de búsqueda al construirlo: aumentar ese trabajo puede mejorar la estructura y encarecer la ingesta. ef, también llamado ef_search en algunas interfaces, controla la amplitud durante una consulta: ampliarla suele mejorar la recuperación de vecinos a costa de tiempo. No es lo mismo que k, que limita los resultados pedidos. Los nombres y restricciones concretos dependen de la implementación. [T4, T6]

Qué muestra nuestra animación

El laboratorio tiene un recorrido fijo por A, D, F y E, con descensos de capa. Sirve para señalar entrada, exploración y refinamiento. Su distancia es euclídea didáctica y su secuencia es determinista. No ejecuta HNSW real ni permite deducir su recall. El índice de Chroma no se inspecciona mediante ese gráfico. [L2]

Error que conviene evitar

Encontrar un nodo donde ya no mejora el paso visible no demuestra haber encontrado el vecino global. La búsqueda real maneja conjuntos de candidatos y visitados; la animación simplifica esas operaciones. No atribuir tiempos ni garantía de exactitud al dibujo.

Diapositiva 8 · Top-k y filtros de metadatos

Integrante 3 · tiempo previsto 2:00

Guion oral dentro del tiempo asignado

Un filtro define quién puede ser candidato; top-k limita cuántos vecinos devolvemos. Pedir joven en el texto no impone una edad máxima.

ABRIR INTERACCIÓN 2. Reiniciar. Mantener $Q=(4,1)$, posición ST, edad máxima 25 y $k=3$: quedan A, F y J. Cambiar k a 8: siguen tres, no se inventan candidatos. Elegir GK y edad 18: no queda ninguno. Reiniciar y mover Q para observar cómo cambia el orden.

La consulta nula no tiene coseno definido: si ambos controles quedan en cero, el módulo pide mover uno. Los símbolos distinguen seleccionados, elegibles y excluidos sin depender sólo del color.

Esta simulación calcula todos los cosenos; no usa ANN. El proyecto real realiza la consulta en Chroma y luego puede reordenar por atributos.

VOLVER A LAS DIAPOSITIVAS y conectar con la administración de esos registros.

Base teórica para preparar esta diapositiva

Un filtro de metadatos expresa una condición sobre campos almacenados. Puede combinar igualdad, rangos y condiciones lógicas. Por ejemplo: entidad = player Y plays_ST = verdadero Y edad $\leq$ 25. El filtro determina elegibilidad; la medida determina cercanía; k indica el máximo de resultados solicitado. Chroma permite expresar condiciones de metadatos con operadores de comparación y combinaciones lógicas. [T8, L2]

Ejemplo resuelto

Supongamos cuatro perfiles: A es ST de 22 años y distancia 0,30; B es CM de 23 y distancia 0,10; C es ST de 29 y distancia 0,20; D es ST de 24 y distancia 0,40. Con ST, edad máxima 25 y $k=3$, sólo A y D son elegibles. Se devuelven esos dos; B no puede entrar aunque sea el más cercano y no se inventa un tercero.

Orden lógico y ejecución física

“Filtrar y elegir vecinos” describe el resultado buscado. El motor puede combinar filtros e índice de distintas formas. Filtrar después de pedir pocos vecinos globales puede dejar menos resultados y omitir elegibles que estaban más abajo. Por eso no debe inferirse la estrategia interna sólo observando la interfaz. En el laboratorio se filtra el conjunto y después se calculan todas las distancias exactamente. [L1, L2]

Distinción que deben poder defender

Combinar vectores con filtros no equivale por sí solo a búsqueda híbrida léxica y densa. Esa búsqueda combinaría señales de recuperación textual y vectorial. Además, escribir “joven” en la frase no impone una edad máxima en esta aplicación: hay que activar el control correspondiente. [L1, L2]

Diapositiva 9 · Persistencia, actualización y escala

Integrante 3 · tiempo previsto 1:30

Guion oral dentro del tiempo asignado

Una base vectorial es más que calcular distancias. Hay que poder conservar la colección, relacionarla con una fuente, actualizarla y restaurarla. Identificadores estables no bastan si quedan perfiles obsoletos. El proyecto usa huella del corpus y versión del perfil para detectar incompatibilidades.

Para estimar componentes float32, multiplicamos N por D por 4 bytes. Con 20.421 vectores y 384 dimensiones son 31,37 MB decimales sólo de componentes. El proceso ocupa más: modelo, grafo, documentos y metadatos.

Particionar reparte registros; replicar agrega copias. El prototipo local no demuestra ninguna de esas capacidades distribuidas. Esas decisiones dependen del gestor y el despliegue.

PASE AL INTEGRANTE 4. Veamos cómo se organizan estas responsabilidades en una solución completa.

Base teórica para preparar esta diapositiva

Persistencia significa conservar los registros para reutilizarlos después de cerrar el proceso. No equivale a disponer de una copia de seguridad ni a tolerar cualquier fallo. Una administración completa debe contemplar altas, consultas, modificaciones, bajas, copias y restauración. [L1]

Si cambia sólo un metadato que no interviene en el texto representado, puede bastar con actualizar ese campo. Si cambia el contenido que se vectoriza, hace falta regenerar su embedding y actualizar el registro. Si cambia el modelo o el modo de construir perfiles, se debe revisar la compatibilidad de toda la colección. En nuestro código, una huella del corpus, una versión y el conteo permiten detectar diferencias y pedir reconstrucción; no hay sincronización automática incremental de cualquier CSV modificado. [L2]

Memoria: cálculo que conviene saber explicar

Para N vectores de D componentes float32, los componentes ocupan aproximadamente $N \times D \times 4$ bytes. Con $20.421 \times 384 \times 4 = 31.366.656$ bytes: unos 31,37 MB decimales, o 29,91 MiB. Esto no mide el consumo total: faltan grafo, documentos, metadatos, estructuras auxiliares y modelo. [L1, L2]

Escala y administración

Escalar verticalmente es agregar recursos a una máquina. Particionar reparte datos entre unidades; replicar mantiene copias. Una copia no reemplaza un backup histórico: una eliminación puede propagarse. Cuantizar reduce la precisión usada para representar componentes y puede ahorrar espacio, a cambio de cambios en la aproximación. Son decisiones generales, no funciones demostradas por nuestro prototipo local. [L1]

Hay que controlar quién consulta, quién modifica y qué registros puede ver cada usuario. Los filtros de la demostración expresan criterios deportivos; no implementan un sistema completo de autorización.

Diapositiva 10 · Arquitectura: ingesta y consulta

Integrante 4 · tiempo previsto 1:40

Guion oral dentro del tiempo asignado

La ingesta ocurre cuando incorporamos o actualizamos fuentes. Validamos, elegimos una unidad de contenido, generamos embeddings y los guardamos. En documentos largos se fragmenta; en nuestro caso principal se construye una ficha por entidad del snapshot. No extraemos reglamentos PDF.

La consulta valida el pedido, genera su embedding, aplica filtros, recupera vecinos y selecciona la evidencia. El modelo de representación debe ser compatible en ambos caminos. La persistencia evita repetir la ingesta en cada búsqueda.

Para diagnosticar rendimiento separamos embedding y API de consulta. `collection.query` incluye filtros y lectura de datos, no sólo navegación del grafo. Reordenamiento y generación son etapas adicionales.

Base teórica para preparar esta diapositiva

La ingesta prepara información para consultas posteriores. Su secuencia conceptual es: leer fuentes, validar y limpiar, elegir unidades de contenido, generar embeddings y guardar registros con trazabilidad. Una unidad demasiado extensa puede mezclar temas; una demasiado pequeña puede perder contexto. En documentos largos suele emplearse fragmentación o chunking. No existe un tamaño universal correcto. [L1]

En nuestro corpus principal se construye un perfil por entidad del snapshot. El texto destinado al embedding describe características y omite nombres y URLs; el documento mostrado conserva identidad y fuente. Esta decisión separa representación de evidencia. No corresponde explicar la demo como si extrajera páginas de un reglamento PDF. [L2]

La consulta tiene otra secuencia: validar el pedido, generar un vector compatible, aplicar condiciones, recuperar candidatos y presentar los resultados. Puede añadirse reordenamiento y, después, generación. Persistir el índice evita volver a vectorizar todo el corpus en cada consulta. [L1, L2]

Ejemplo para explicar la arquitectura

“La ingesta se parece a preparar el catálogo de una biblioteca. Consultar es usar ese catálogo. No volvemos a catalogar todos los libros cada vez que alguien pide uno.” La analogía explica la separación de tareas; no implica que el índice sea un catálogo alfabético.

Rendimiento y trazabilidad

Separar tiempos permite ubicar un cuello de botella. Medir `collection.query` no mide únicamente saltos del grafo: también intervienen filtros y lectura de resultados. Hay que registrar modelo, configuración, datos y condiciones del ensayo. Para verificar una respuesta, conservar ID, fuente y fecha junto a la ficha. [L2]

Diapositiva 11 · Tecnologías y aplicaciones

Integrante 4 · tiempo previsto 1:40

Guion oral dentro del tiempo asignado

Chroma es la base usada en este proyecto. Qdrant es otra alternativa especializada. pgvector permite integrar vectores en PostgreSQL. Elegir depende de infraestructura, filtros, volumen, personal y controles, no sólo de una lista de funcionalidades.

Las aplicaciones incluyen búsquedas, recomendaciones, detección de contenido parecido y RAG. Tres casos permiten distinguir funciones: LinkedIn estudia cómo conservar relaciones mediante grafos de conocimiento; DoorDash agrega control de calidad de respuestas; Spotify desarrolla Voyager para vecinos próximos sin exigir generación.

No trasladamos las métricas de esas empresas a nuestro buscador. Tampoco una biblioteca como Voyager reemplaza por sí sola todas las funciones de un gestor.

TRANSICIÓN. Ahora podemos ubicar la base vectorial dentro de RAG sin confundir los dos conceptos.

Base teórica para preparar esta diapositiva

Una biblioteca de vecinos se ocupa principalmente de indexar y buscar representaciones. Un gestor agrega operaciones y administración de datos. Una plataforma completa puede sumar servicios de despliegue y operación. Antes de comparar productos hay que identificar qué responsabilidades cubre cada uno. [L1]

En esta solución, Chroma administra la colección; MiniLM produce los embeddings; Streamlit muestra la interfaz y Python coordina el proceso. pgvector incorpora búsqueda de similitud a PostgreSQL y admite búsqueda exacta e índices aproximados. Su existencia permite explicar la complementariedad con datos relacionales. Qdrant es otra alternativa tratada en el informe; no fue el motor probado en esta demo. [T5, L1, L2]

Cómo conectar tecnología y necesidad

Para elegir una solución hay que conocer cantidad de vectores, dimensión, crecimiento, frecuencia de actualización, condiciones de consulta y requisitos de operación. Después se compara rendimiento con la misma carga y calidad requerida. Una lista de funcionalidades no reemplaza ese ensayo. [L1]

La recuperación por similitud puede usarse para localizar documentos, encontrar productos parecidos, agrupar contenido o proponer candidatos para recomendación. Con textos, imágenes o audio se necesitan representaciones adecuadas a cada tarea. Recomendación implica además criterios de negocio, disponibilidad y evaluación: cercanía sola no resuelve todo el problema. [L1]

Qué decir sobre los casos del informe

Usar los casos empresariales como ejemplos de decisiones de arquitectura y evaluación. No presentar sus resultados como mediciones del buscador. En especial, una biblioteca de vecinos y una solución de respuestas con evidencia no son el mismo tipo de componente. Las fuentes particulares de esos casos permanecen en las notas originales. [L1]

Diapositiva 12 · Dónde participa la base vectorial en RAG

Integrante 4 · tiempo previsto 1:40

Guion oral dentro del tiempo asignado

RAG combina recuperación con generación. El recuperador selecciona evidencia externa y el generador la recibe en el contexto. Esa recuperación puede ser vectorial, léxica o combinada.

El LLM no lee automáticamente toda la base. Sólo recibe el material que le enviamos. Por eso puede responder con contexto insuficiente, interpretar mal una ficha o agregar información no sustentada. Pedirle que cite no es una validación automática.

Sin generador, nuestro sistema sigue siendo un buscador vectorial con reordenamiento. Con generador, puede producir una comparación apoyada en perfiles. Si usamos una API remota, consulta y perfiles seleccionados salen del equipo aunque los embeddings sean locales.

PASE AL INTEGRANTE 5. Ahora mostramos qué implementamos realmente y qué se puede verificar en la aplicación.

Base teórica para preparar esta diapositiva

RAG significa Retrieval-Augmented Generation: generación aumentada mediante recuperación. La idea es proporcionar al generador información externa recuperada para la consulta. Se separa el conocimiento almacenado en parámetros del modelo del material consultable que se aporta como contexto. El artículo original combina componentes de recuperación y generación; las aplicaciones actuales pueden organizarlos de diferentes maneras. [T7]

En el esquema de nuestra aplicación, primero se recuperan perfiles, después se prepara el contexto y sólo entonces un generador opcional redacta una respuesta. La base vectorial interviene en la recuperación; no redacta. El LLM no recibe toda la colección por el hecho de estar conectado al sistema. La recuperación también podría ser léxica o combinar varias señales. [L1, L2]

Ejemplo para explicarlo con palabras propias

“Si queremos un informe comparativo, buscamos las fichas y se las entregamos al modelo junto con la pregunta. La respuesta debería apoyarse en esas fichas. Si buscamos sin generador, podemos mostrar directamente la evidencia y explicar nosotros la comparación.”

Límites y administración

Una respuesta puede fallar porque se recuperaron documentos equivocados, porque el contexto quedó incompleto o porque el generador interpretó mal la evidencia. Agregar recuperación no garantiza eliminar invenciones. Deben evaluarse por separado los resultados recuperados y la fidelidad de lo redactado. Un texto recuperado es una fuente de datos; no debe adquirir autoridad para cambiar las instrucciones del sistema. [L1]

Si se envía el contexto a una API remota, salen del equipo los datos incluidos en esa solicitud. Embeddings locales no garantizan procesamiento completamente local. Con generador none, esta demostración comprueba recuperación y reordenamiento sin generar un reporte LLM. [L2]

Diapositiva 13 · Nuestro buscador de jugadores

Integrante 5 · tiempo previsto 1:20

Guion oral dentro del tiempo asignado

El caso usa Python, Streamlit, Chroma y MiniLM local. La colección principal contiene 20.421 fichas: 18.350 jugadores, 702 equipos y 1.369 técnicos. Son datos de FIFA/EA FC edición 24, actualización 2, fechados el 22 de septiembre de 2023. No son estadísticas oficiales de partidos ni planteles actuales.

Primero se recupera un pool de vecinos, con un límite de 5.000. Si la consulta reconoce atributos, se reordena por percentiles; los empates usan valoraciones originales y luego similitud. Para regate no confiamos en que un embedding compare números.

El filtro de posición puede hacer que el pool cubra todos los elegibles. Sin ese filtro puede quedar cobertura parcial. Esto limita quién llega al ranking final.

Base teórica para preparar esta diapositiva

El caso práctico tiene tres representaciones que no deben confundirse: la ficha legible del jugador, su embedding semántico de 384 componentes y los atributos numéricos de los metadatos. El vector no es una lista de las 384 mejores estadísticas ni se usa como sustituto de las valoraciones originales. [L2]

La colección contiene 20.421 perfiles: 18.350 jugadores, 702 equipos y 1.369 técnicos. Son registros del snapshot FIFA/EA FC 24, actualización 2, con fecha 22/09/2023. La consulta de la demo filtra jugadores que admiten ST y excluye arqueros; el conjunto resultante tiene 3.194 registros en la evidencia guardada. No se está buscando únicamente entre las cinco personas mostradas. [L2, L3]

Por qué se agrega reordenamiento

Representar una frase por similitud no garantiza comparar correctamente números como regate 92 y regate 41. El código reconoce términos de atributos y, si hay datos disponibles, ordena el pool por el promedio de sus percentiles. Un percentil ubica un valor con respecto al grupo de referencia; no es una probabilidad de rendimiento deportivo. [L2]

Los empates se resuelven con valoraciones originales y luego similitud. Cuando no se reconoce un atributo utilizable, se conserva el orden vectorial. El reordenador trabaja sólo con candidatos recuperados; no puede rescatar un jugador ausente del pool. El límite de recuperación es 5.000. [L2]

Idea que debe quedar clara

Esta aplicación permite explicar la colaboración entre representación semántica, restricciones exactas y ordenamiento numérico. Si la necesidad fuera únicamente ordenar todos los delanteros por regate, una consulta estructurada podría resolverla sin embeddings. [L1, L2]

Diapositiva 14 · Demostración: delantero con regate

Integrante 5 · tiempo previsto 3:20

Guion oral dentro del tiempo asignado

ABRIR EL BUSCADOR desde el botón del laboratorio. Seleccionar CSV del usuario, activar el buscador, entidad player, posición ST, cinco resultados, generador none y reordenamiento activo. Dejar edad sin límite, liga Todas y regate mínimo cero. Consultar Delantero con regate y desborde.

Mostrar colección y dimensión. Abrir Ordenamiento aplicado: la consulta reconoce dribbling y el pool cubre los 3.194 elegibles. Identificar distancia y ajuste por atributos. Mbappé puede estar primero aunque Griezmann tenga menor distancia coseno: el ranking final depende de regate.

Desactivar reordenamiento y repetir la misma consulta para mostrar la diferencia de criterio, sin prometer un listado específico. Volver a activarlo. No activar un proveedor remoto durante la demo si no se ensayó.

RESPALDO: esta tabla conserva una consulta registrada el 12/09/2026. Si el equipo tarda, usarla y aclarar que no es una ejecución en vivo. No atribuirle las métricas del antiguo ejemplo de normativa universitaria.

Base teórica para interpretar la demostración

Cada control responde a una pregunta diferente: la posición determina elegibilidad; la consulta orienta la recuperación y permite reconocer atributos; k fija cuántos se muestran; el reordenamiento decide el criterio final entre candidatos; none desactiva la generación. Nombrarlos antes de ejecutar evita atribuir todo a una supuesta decisión de la inteligencia artificial. [L2]

Lectura de los resultados registrados

En la consulta guardada, Mbappé tiene regate 92 y distancia aproximada 0,5458; Griezmann tiene regate 88 y distancia 0,4895. Por distancia coseno, Griezmann está más cerca. Por el criterio final de regate, Mbappé queda primero. No hay contradicción: se están utilizando dos órdenes distintos. [L3]

Score = 1 - distancia permite expresar similitud en lugar de distancia. No convierte la medida en confianza. “Ajuste atributos” resume percentiles y “valoración media” resume los valores originales usados en el desempate. Ninguno representa la probabilidad de que un fichaje sea exitoso. [L2]

Cobertura y límites de lo demostrado

En esta evidencia, el pool recuperó los 3.194 elegibles: el orden por atributos pudo considerar a todo ese subconjunto. En una consulta más amplia, el límite de 5.000 puede dejar candidatos afuera. Por eso no se promete que todo ranking final examine siempre el corpus entero. [L3]

Desactivar reordenamiento muestra qué cambia al usar sólo distancia. Las diferencias de orden sirven para enseñar criterios; no prueban por sí solas que un embedding sea bueno o malo para toda consulta. Tampoco una medición aislada permite comparar tecnologías. La búsqueda LIKE de la aplicación es un escaneo léxico en pandas, no un experimento contra un servidor SQL. [L1, L2]

Frase útil al cerrar la demo

“Pudimos seguir el camino desde la consulta hasta la ficha original. Eso hace verificable el resultado. La interpretación deportiva depende de la calidad y vigencia de la fuente.”

Diapositiva 15 · Conclusiones y límites

Integrante 5 · tiempo previsto 1:20

Guion oral dentro del tiempo asignado

CERRAR CON TRES IDEAS. Primero, los embeddings vuelven comparables las representaciones, pero no garantizan relevancia. Segundo, el índice aproxima vecinos y los filtros verifican condiciones; no son la misma responsabilidad. Tercero, administrar fuentes, modelos, permisos y evaluación es parte de operar la solución.

Nuestro buscador funciona con recuperación local y ranking por atributos. Su evidencia no demuestra superioridad general frente a SQL ni calidad profesional de scouting. Los datos son históricos, el pool puede limitar cobertura y todavía falta una evaluación etiquetada de relevancia.

RAG es una aplicación opcional de esa recuperación. Si alguien pregunta por el reporte LLM, mostrar dónde se configura y explicar que esta demostración principal se ejecutó sin generación.

CIERRE. Podemos verificar cómo funciona el mecanismo y también reconocer hasta dónde llega la evidencia. Abrir preguntas.

Base teórica para construir la conclusión

La utilidad de una base vectorial depende de varias decisiones: qué contenido se representa, con qué modelo, cómo se mide cercanía, qué índice se utiliza, qué condiciones se aplican y cómo se administra la colección. Un fallo en una etapa puede limitar el sistema aunque las otras funcionen correctamente. [L1]

Cómo evaluar la recuperación

Con juicios de relevancia, Precision@k indica qué proporción de los primeros k resultados es pertinente. Si cuatro de cinco son relevantes, $P@5 = 4/5 = 0,80$. Recall@k indica qué proporción de todos los relevantes conocidos se recuperó: si existen diez relevantes y aparecen cuatro, $R@5 = 4/10 = 0,40$. MRR promedia el inverso de la posición del primer resultado relevante en cada consulta. Si aparece primero, aporta 1; si aparece tercero, $1/3$. [L1]

El recall de vecinos ANN se compara contra un top-k exacto del mismo espacio. Es distinto del recall de relevancia humana. Para medir rendimiento se necesitan consultas representativas, repeticiones, condiciones comparables y registro de latencia y recursos. Si se usa generación, se agrega evaluación de respaldo factual y correspondencia con las fuentes. [L1]

Qué permite concluir nuestro trabajo

La evidencia demuestra una implementación funcional de recuperación, filtros y reordenamiento sobre un corpus identificado. Las pruebas de software verifican comportamientos, no equivalen a una evaluación exhaustiva de pertinencia. Las valoraciones históricas de un videojuego tampoco validan una recomendación de scouting actual. [L2, L3]

Cierre oral sugerido

“Las bases vectoriales incorporan consultas por cercanía a los sistemas de información. Su valor depende de la representación, de la administración de los datos y de una evaluación adecuada. Nuestro buscador muestra cómo integrarlas con filtros y atributos exactos, y permite incorporar generación como una etapa opcional.”

Preguntas que puede hacer la cátedra

¿Una base vectorial reemplaza a SQL?

No. Las consultas exactas, relaciones e integridad siguen siendo valiosas. Se puede integrar búsqueda vectorial en un gestor relacional mediante una extensión como pgvector.

¿Qué diferencia hay entre un vector y un embedding?

Un vector es una lista numérica. Un embedding es una representación del contenido producida por un modelo. Sus componentes no suelen ser atributos humanos interpretables.

¿Por qué usan 384 dimensiones?

Es la dimensión de salida de MiniLM en la función de embeddings utilizada. No la elegimos como cantidad de estadísticas futbolísticas. Cambiar modelo puede cambiar el espacio y exige reindexar.

¿Coseno 0,9 significa 90 % de acierto?

No. Es similitud angular. No es una probabilidad, y debe interpretarse según el modelo y la tarea. Distancia coseno = $1 - \text{similitud}$.

¿Por qué no comparan todos los vectores?

La búsqueda exacta es válida, especialmente en corpus pequeños. ANN reduce exploración al crecer el volumen, a cambio de posibles omisiones. Para evaluar ANN hay que compararlo con vecinos exactos del mismo espacio.

¿El grafo mostrado es el de Chroma?

No. Es una simulación fija de varias capas, con distancia euclídea simple. Explica entrada, exploración y descenso. Chroma utiliza HNSW y puede tener un buffer de fuerza bruta; no estamos inspeccionando esas conexiones internas.

¿Por qué Mbappé queda primero si otro tiene menor distancia?

Porque el orden final se calcula con atributos reconocidos en la consulta. La búsqueda vectorial elige el pool; el reordenador prioriza percentiles, desempata por valoraciones originales y luego por similitud. Desactivar reordenamiento muestra el otro criterio.

¿Entonces para qué usar vectores si ya tienen regate?

Para recuperar perfiles a partir de descripciones variadas. Una consulta puramente estructurada como ordenar todos los delanteros por regate se puede resolver directamente sin embeddings. La contribución educativa es mostrar la integración y sus límites, no justificar vectores para toda consulta.

¿Un filtro escrito en la pregunta se aplica automáticamente?

No. Los controles de posición, liga, edad y regate expresan condiciones exactas. El texto orienta recuperación y detección de atributos, pero no crea esos filtros por sí solo.

¿Qué pasa con un jugador que queda fuera del pool?

El reordenador no puede recuperarlo. El tope es 5.000 candidatos. Si los filtros hacen que el pool incluya todos los elegibles, la comparación por atributos cubre ese subconjunto; de otro modo la cobertura es parcial.

¿El buscador reconoce cualquier pregunta sin respuesta?

No tiene un umbral de relevancia calibrado para abstenerse. Evita generar cuando no hay candidatos por los filtros, pero una consulta ajena al dominio puede recibir vecinos poco pertinentes.

¿Todo funciona sin Internet?

Las interacciones sí. La recuperación también puede funcionar localmente si modelo e índice ya están preparados. La descarga inicial del modelo y un proveedor LLM remoto requieren conectividad. Ollama exige un servicio y un modelo local preparados previamente.

¿Dónde está RAG si el generador está en none?

En ese modo mostramos recuperación y evidencia, sin generación. El pipeline permite agregar un proveedor y enviar perfiles como contexto. Para mostrar RAG completo habría que habilitar y probar previamente ese generador.

¿Los datos son actuales y oficiales?

Son valoraciones del videojuego FIFA/EA FC, edición 24, actualización 2, fechadas el 22/09/2023. No son estadísticas oficiales del Mundial de Clubes 2025 ni información deportiva actual.

¿Demostraron que ANN es más rápido o mejor que LIKE?

No. La comparación implementa un escaneo léxico en pandas con OR de términos, no un motor SQL. Las mediciones dependen de la ejecución, la cantidad de perfiles retornados y la carga del equipo. Falta un protocolo de rendimiento y relevancia comparable.

¿Cómo evaluarían la calidad?

Con consultas y relevancia etiquetadas para medir Precision@K, Recall@K y MRR. Por separado, mediríamos recuperación de vecinos exactos y fidelidad de respuestas generadas. Una prueba funcional no sustituye esas evaluaciones.

Ruta de respaldo ante fallos

Si falla Streamlit: abrir Laboratorio_Vectorial_OFFLINE.html y continuar con la teoría. El archivo es autocontenido; no necesita instalación.

Si falla la interacción: usar las capturas de respaldo de las diapositivas 5, 7 y 8. Describir los cambios previstos de ángulo, filtros y capas. Las capturas completas también están en respaldo/.

Si el buscador tarda más de 20 segundos: pasar a la tabla registrada de la diapositiva 14. Explicar que corresponde a una ejecución previa del 12/09/2026. No simular que el resultado guardado acaba de producirse.

Si falla PowerPoint: abrir Bases_Vectoriales_Exposicion_Final.pdf. Las imágenes y tablas necesarias para seguir la exposición están incluidas.

Si falla la red: mantener none y usar la recuperación ya preparada. No dedicar tiempo de exposición a configurar un proveedor.

Si necesitan recuperar tiempo: reducir ejemplos verbales, recorrer sólo los pasos 1, 3, 5 y 7 de HNSW y omitir repetir la búsqueda sin reordenamiento. Conservar definiciones, distinción de fuentes y conclusión.

Ensayo de los cinco integrantes

Hacer un ensayo grupal cronometrado antes de entregar. Usar un temporizador visible sólo para quienes exponen y anotar los tiempos reales por bloque. La distribución de 26 minutos es un presupuesto de guion, no una medición de un ensayo humano.

Al minuto 4 debe comenzar el integrante 2; al 9, el 3; al 15, el 4; al 20, el 5. Ensayar un segundo recorrido usando sólo PDF y capturas. Cada integrante debe poder explicar qué muestran los gráficos y reconocer que son didácticos.

No memorizar resultados exactos de tiempos ni prometer el mismo ranking vectorial en otra versión del corpus. Ensayar con el snapshot conservado y mantener el sentido de cada métrica.

Referencias de los fundamentos ampliados

L1. Informe revisado del proyecto: TPI_Bases_Vectoriales_Futbol_REVISADO.pdf, secciones 1-5, 7-9. Archivo local en output/pdf. Fuente principal para el alcance y los conceptos de esta exposición.

L2. Código comprobado del proyecto: vector_store.py (colección, ingesta y consulta), reranker.py (detección de atributos y orden), supplied_app.py (controles), rag_pipeline.py

(generación opcional) y `exposition_assets/` (modelos didácticos). La implementación concreta prevalece al describir la demo.

L3. Evidencia de consulta registrada el 12/09/2026: `respaldo/consulta_registrada_2026-09-12.json`. Corpus histórico del 22/09/2023. No es un benchmark general ni una ejecución en vivo.

T1. Sentence Transformers. Ficha oficial de `all-MiniLM-L6-v2`: dimensión y uso del modelo. <https://huggingface.co/sentence-transformers/all-MiniLM-L6-v2>

T2. Manning, Raghavan y Schütze (2008). Introduction to Information Retrieval, sección sobre producto interno, normas y coseno. <https://nlp.stanford.edu/IR-book/html/htmledition/dot-products-1.html>

T3. Malkov y Yashunin (2016; versión revisada 2018). Efficient and robust approximate nearest neighbor search using Hierarchical Navigable Small World graphs. <https://arxiv.org/abs/1603.09320>

T4. hnswlib. Documentación de parámetros del algoritmo: `M`, `ef_construction` y `ef`. https://github.com/nmslib/hnswlib/blob/master/ALGO_PARAMS.md

T5. pgvector. Documentación oficial de búsqueda vectorial en PostgreSQL. <https://github.com/pgvector/pgvector>

T6. Chroma. Configuración de colecciones e índices. <https://docs.trychroma.com/docs/collections/configure>

T7. Lewis y colaboradores (2020). Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks. <https://arxiv.org/abs/2005.11401>

T8. Chroma. Filtros de metadatos. <https://docs.trychroma.com/docs/querying-collections/metadata-filtering>

Documentación consultada el 12/09/2026. Los cálculos y perfiles numéricos de práctica son ejemplos didácticos elaborados para este guion; no son resultados adicionales del corpus.

Fuentes del guion y de las diapositivas

Manning et al. (2008). <https://nlp.stanford.edu/IR-book/>

Proyecto local: `vector_store.py`, `supplied_data.py`, `rag_pipeline.py` y `reranker.py`.

pgvector. <https://github.com/pgvector/pgvector>

Reimers y Gurevych (2019). <https://arxiv.org/abs/1908.10084>

Sentence Transformers. <https://huggingface.co/sentence-transformers/all-MiniLM-L6-v2>

Informe revisado, secciones 2–4; modelo geométrico de elaboración propia.

Captura de la interacción local, estado de 90 grados; respaldo estático.

Malkov y Yashunin (2020). <https://arxiv.org/abs/1603.09320>

Captura del laboratorio didáctico, paso 6 de 7.

Captura de perfiles sintéticos: ST, edad máxima 25, k=3.

Chroma. <https://cookbook.chromadb.dev/core/collections/>

Cálculo propio: $20.421 \times 384 \times 4 = 31.366.656$ bytes.

Chroma: <https://docs.trychroma.com/>

Qdrant: <https://qdrant.tech/documentation/>

pgvector: <https://github.com/pgvector/pgvector>

Xu et al. (2024): <https://arxiv.org/abs/2404.17723>

Jia et al. (2024): <https://careersatdoordash.com/blog/large-language-modules-based-dash-e-r-support-automation/>

Spotify: <https://github.com/spotify/voyager>

Lewis et al. (2020). <https://arxiv.org/abs/2005.11401>

data/supplied/manifest.json

output/pdf/evidencia_busqueda.json; consulta registrada el 12/09/2026, sin LLM.